

Parte 2. Comprensión del flujo colaborativo

Control de Actividades - Desarrollo de Aplicaciones

1. Flujo colaborativo

El orden lógico sería más o menos así. Primero haces Fork, que es sacar una copia del repositorio a tu propia cuenta de GitHub. Luego haces Clone, que es bajar esa copia a tu computadora. Después creas un Branch, o sea una rama nueva para no trabajar directo en la rama principal. Ahí ya puedes Modificar archivos, que es donde escribes o corriges el código. Cuando terminas una parte haces Commit, que guarda esos cambios en tu historial con un mensaje. Luego haces Push, que sube esos commits a tu repositorio en GitHub (tu fork). Con eso ya puedes abrir un Pull Request, que es pedirle al dueño del proyecto original que revise tus cambios. El dueño hace el Review, o sea revisa el código. Y si está bien, se hace el Merge, que es cuando tus cambios se unen finalmente a la rama principal del repositorio original.

Yo agregaría también el Pull (que no es lo mismo que Pull Request). Este sirve para traer a tu computadora los cambios que ya están en el repositorio remoto, por ejemplo si alguien más subió algo nuevo y tú todavía no lo tienes.

2. Fork y Clone

Esa afirmación no es correcta. Clone no tiene nada que ver con tu cuenta de GitHub, lo que hace es descargar el repositorio a tu computadora. El que crea una copia dentro de tu cuenta de GitHub es el Fork, no el Clone.

Entonces la diferencia sería: Fork copia el proyecto a tu cuenta de GitHub, o sea queda en la nube. Clone lo que hace es bajar un repositorio (puede ser el original o tu fork) desde GitHub hasta tu computadora, para que puedas trabajar en él desde tu máquina.

3. Pull Request

No, todavía no forman parte del repositorio original. Con el Push lo único que pasa es que tus cambios quedan en tu propio fork, dentro de tu cuenta de GitHub. El dueño del repositorio original ni se entera todavía.

Para que esos cambios lleguen al repositorio original hay que abrir un Pull Request. Ahí el dueño del proyecto revisa lo que hiciste y, si le parece bien, hace Merge para juntar tus cambios con la rama principal.

4. Request Changes

Cuando pasa esto, lo que tengo que hacer es corregir lo que el dueño pidió, usando la misma rama que ya tenía. Después hago commit otra vez y push otra vez.

No hace falta crear otro Pull Request. El que ya existe sigue conectado a esa rama, entonces cuando hago push de nuevo los commits nuevos se agregan solos al mismo Pull Request, y

queda listo para que lo revisen otra vez.

5. Merge y repositorio local

Pasa porque el Merge se hizo en GitHub, o sea en el repositorio remoto, pero eso no baja solo a la computadora del dueño. Git no actualiza tu máquina en automático solo porque algo cambió en la nube.

Para arreglar eso, el dueño tiene que pararse en su rama principal (como main) y hacer git pull. Con eso ya trae los cambios de GitHub a su computadora.

6. Sync Fork

En este caso usaría la opción Sync Fork que tiene GitHub. Esto trae los commits nuevos del repositorio original y los mete en mi fork, para que quede al día.

La diferencia con git pull es hacia dónde va la actualización. Sync Fork actualiza mi fork, que está en GitHub, trayendo lo nuevo del repositorio original. En cambio git pull actualiza mi computadora, trayendo lo que ya está en un remoto, casi siempre mi propio fork.

Por eso si quiero tener todo actualizado en mi computadora, primero debería hacer Sync Fork para que mi fork quede al día, y después git pull para bajar eso a mi máquina.